

当前匿名凭证 Demo 操作说明

本文档描述当前仓库里已经跑通的匿名展示 demo:

- 目录: `lib/examples/mdoc_anoncred/`
- 输入: 真实 `mdoc DeviceResponse` 样本
- 证明: 调用 `run_mdoc_prover(...)`
- 验证: 调用 `run_mdoc_verifier(...)`
- challenge: 由 verifier 现场生成新的 OpenID4VP `SessionTranscript`
- 持有者绑定: holder 在每次 `prove` 时对本次 transcript 动态 device signing

这份 demo 的目标不是完整钱包系统, 而是先把下面这条本地文件调用链跑通:

```
issuer issue -> verifier request -> holder prove -> verifier verify
```

1. 这个 demo 现在能做什么

当前已经实现的基本作用:

1. `issuer` 从真实 `mdoc` 样本出发, 物化一份 holder 可用的凭证目录
2. `issuer` 会生成新的 issuer key 和 holder device key, 并把它们补丁进真实 `mdoc` 模板
3. `verifier request` 会生成新的 OpenID4VP `SessionTranscript`
4. `holder prove` 会对这次 transcript 实时签名, 再生成零知识证明
5. `verifier verify` 只根据 `issuer_public + request + presentation` 做验证

当前支持的 claim alias:

- `age_over_18`
- `family_name_mustermann`
- `birth_date_1971_09_01`
- `height_175`

另外, 当前还新增了一条最小自定义签发链路:

- `issue-custom`

它会交互式输入并现场签发一个最小真实 `mdoc`, 目前只支持:

- `family_name`
- `given_name`
- `birth_date`
- `issue_date`
- `expiry_date`
- `issuing_country`
- `age_over_18`

2. 当前 demo 的目录和角色

当前 CLI 分成 3 个角色：

- `mdoc_anoncred_issuer`
- `mdoc_anoncred_holder`
- `mdoc_anoncred_verifier`

另外新增了一个交互式总入口：

- `mdoc_anoncred_console`

它们分别对应：

1. `issuer`
2. `holder`
3. `verifier`

3. 编译

在仓库根目录执行：

```
1 CXX=clang++ cmake -D CMAKE_BUILD_TYPE=Release -S lib -B build-demo-cli
2 cmake --build build-demo-cli -j 16 --target \
3     mdoc_anoncred_issuer \
4     mdoc_anoncred_holder \
5     mdoc_anoncred_verifier \
6     mdoc_anoncred_demo_test
```

编译完成后，可执行文件在：

- `./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_console`
- `./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_issuer`
- `./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_holder`
- `./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier`

4. 交互式一键运行

如果你想让用户直接走交互式流程，最简单的方式是直接运行：

```
1 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_console guided \
2     --out-root run/mdoc-interactive
```

它现在会先提示：

1. 选择签发模式
2. 如果选样本模式：选择 mdoc 样本
3. 如果选自定义模式：输入字段值
4. 选择 1 到 2 个 claim

然后自动执行：

```
issue -> request -> prove -> verify
```

执行成功时会输出：

- `verification ok: ...`
- `issue dir: ...`
- `request dir: ...`
- `presentation dir: ...`

当前交互式模式的边界：

- 样本模式：基于真实预置 mdoc 样本
- 自定义模式：已经支持输入固定 schema 的真实 mdoc
- 还不是任意 schema 的通用 mdoc issuance builder

5. 分步运行流程

下面是一套可以直接复制的最短命令：

```
1  mkdir -p run/mdoc-demo
2
3  ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_issuer issue \
4  --example 3 \
5  --out run/mdoc-demo/issue
6
7  ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier request \
8  --issuer-public run/mdoc-demo/issue/issuer_public \
9  --claim age_over_18 \
10 --out run/mdoc-demo/request
11
12 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_holder prove \
13 --holder run/mdoc-demo/issue/holder \
14 --issuer-public run/mdoc-demo/issue/issuer_public \
15 --request run/mdoc-demo/request \
16 --out run/mdoc-demo/presentation
17
18 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier verify \
19 --issuer-public run/mdoc-demo/issue/issuer_public \
20 --request run/mdoc-demo/request \
21 --presentation run/mdoc-demo/presentation
```

验证成功时会输出类似：

```
1  verification ok: age_over_18
```

如果一次请求多个 claim，例如：

```
1  ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier request \
2  --issuer-public run/mdoc-demo/issue/issuer_public \
3  --claim age_over_18 \
4  --claim family_name_mustermann \
5  --out run/mdoc-demo/request
```

验证成功时会输出类似：

```
1 | verification ok: age_over_18, family_name_mustermann
```

6. 自定义交互式签发

如果你想让用户自己输入字段，再现场签发最小 mdoc，可以使用：

```
1 | ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_issuer issue-custom \  
2 |   --out run/mdoc-custom/issue
```

它会提示输入：

1. family_name
2. given_name
3. birth_date (YYYY-MM-DD)
4. issue_date (YYYY-MM-DD)
5. expiry_date (YYYY-MM-DD)
6. issuing_country (2 letters)
7. age_over_18 (true/false)

直接回车会使用默认值。

一条完整的自定义链路如下：

```
1 | ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_issuer issue-custom \  
2 |   --out run/mdoc-custom/issue  
3 |  
4 | ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier request \  
5 |   --issuer-public run/mdoc-custom/issue/issuer_public \  
6 |   --claim given_name \  
7 |   --claim issuing_country \  
8 |   --out run/mdoc-custom/request  
9 |  
10 | ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_holder prove \  
11 |   --holder run/mdoc-custom/issue/holder \  
12 |   --issuer-public run/mdoc-custom/issue/issuer_public \  
13 |   --request run/mdoc-custom/request \  
14 |   --out run/mdoc-custom/presentation  
15 |  
16 | ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier verify \  
17 |   --issuer-public run/mdoc-custom/issue/issuer_public \  
18 |   --request run/mdoc-custom/request \  
19 |   --presentation run/mdoc-custom/presentation
```

成功输出类似：

```
1 | verification ok: given_name, issuing_country
```

7. 每一步会产出什么

7.1 issuer issue

命令:

```
1 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_issuer issue \  
2   --example 3 \  
3   --out run/mdoc-demo/issue
```

产物:

- run/mdoc-demo/issue/holder/
- run/mdoc-demo/issue/issuer_public/

其中:

holder/ 里主要有:

- device_response.cbor
- device_sk.txt
- device_pkx.txt
- device_pky.txt
- doc_type.txt

issuer_public/ 里主要有:

- issuer_pkx.txt
- issuer_pky.txt
- doc_type.txt
- now.txt
- client_id.txt
- response_uri.txt

含义:

1. holder/ 是 holder prove 需要的私有材料
2. issuer_public/ 是 verifier request / verify 需要的公开材料

交互方式:

- 如果不传 --example, issuer issue 会提示用户选择样本

7.2 verifier request

命令:

```
1 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier request \  
2   --issuer-public run/mdoc-demo/issue/issuer_public \  
3   --claim age_over_18 \  
4   --out run/mdoc-demo/request
```

产物:

- `reader_request.cbor`
- `session_transcript.cbor`
- `openid4vp_request.json`

含义:

1. `session_transcript.cbor` 是本次验证会话的真实 transcript
2. `reader_request.cbor` 是当前 demo 的主请求编码文件
3. `openid4vp_request.json` 是便于查看和对接的 OpenID4VP 风格请求描述

注意:

- 每次执行 `request`, 都会生成新的 transcript
- 同一个 holder 不能把旧 presentation 复用于新的 request

交互方式:

- 如果不传 `--claim`, `verifier request` 会提示用户从当前样本支持的 claim 里选择 1 到 2 个

7.3 holder prove

命令:

```
1 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_holder prove \  
2   --holder run/mdoc-demo/issue/holder \  
3   --issuer-public run/mdoc-demo/issue/issuer_public \  
4   --request run/mdoc-demo/request \  
5   --out run/mdoc-demo/presentation
```

这一步会做 3 件事:

1. 读取 holder 侧保存的真实 `DeviceResponse`
2. 读取本次 `session_transcript.cbor`
3. 用 holder 的 device 私钥对这次 transcript 动态签名, 然后调用 `run_mdoc_prover(...)`

产物:

- `run/mdoc-demo/presentation/proof.bin`
- `run/mdoc-demo/presentation/claim_alias_0.txt`
- `run/mdoc-demo/presentation/claims_count.txt`

7.4 verifier verify

命令:

```
1 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier verify \  
2   --issuer-public run/mdoc-demo/issue/issuer_public \  
3   --request run/mdoc-demo/request \  
4   --presentation run/mdoc-demo/presentation
```

这一步会调用 `run_mdoc_verifier(...)`。

验证成功时：

- 退出码为 0
- 输出 `verification ok: ...`

验证失败时：

- 退出码非 0
- 输出 `verification failed: ...` 或 `verify failed: ...`

7. 多 claim 示例

```
1 mkdir -p run/mdoc-demo-2
2
3 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_issuer issue \
4   --example 3 \
5   --out run/mdoc-demo-2/issue
6
7 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier request \
8   --issuer-public run/mdoc-demo-2/issue/issuer_public \
9   --claim age_over_18 \
10  --claim family_name_mustermann \
11  --out run/mdoc-demo-2/request
12
13 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_holder prove \
14   --holder run/mdoc-demo-2/issue/holder \
15   --issuer-public run/mdoc-demo-2/issue/issuer_public \
16   --request run/mdoc-demo-2/request \
17   --out run/mdoc-demo-2/presentation
18
19 ./build-demo-cli/examples/mdoc_anoncred/mdoc_anoncred_verifier verify \
20   --issuer-public run/mdoc-demo-2/issue/issuer_public \
21   --request run/mdoc-demo-2/request \
22   --presentation run/mdoc-demo-2/presentation
```

8. 运行测试

执行：

```
1 ctest --test-dir build-demo-cli -R 'MdocAnoncredDemo\.' --output-on-failure
```

当前测试覆盖：

1. 单 claim round-trip
2. 双 claim round-trip
3. tampered proof 失败
4. request transcript freshness
5. presentation 不能跨 request 重放

9. 目前这个 demo 的边界

这套 demo 已经具备：

1. 真实 mdoc 结构输入
2. verifier 现场生成 transcript
3. holder 动态 device signing
4. 真实 `run_mdoc_prover` / `run_mdoc_verifier` 调用链

但它仍然是一个本地文件 demo，不是完整生产系统。

目前还没有做的事情包括：

1. 完整网络版 OpenID4VP / ISO `DeviceRequest` 交互
2. 完整 wallet / reader 互操作
3. 标准化 issuer issuance service
4. revocation / status / trust chain 全流程

10. 一句话理解当前实现

当前你已经实现的是：

一个基于真实 mdoc 样本、由 verifier 现场生成 challenge、由 holder 动态签名并生成 ZK proof、再由 verifier 验证的最小匿名展示 demo。